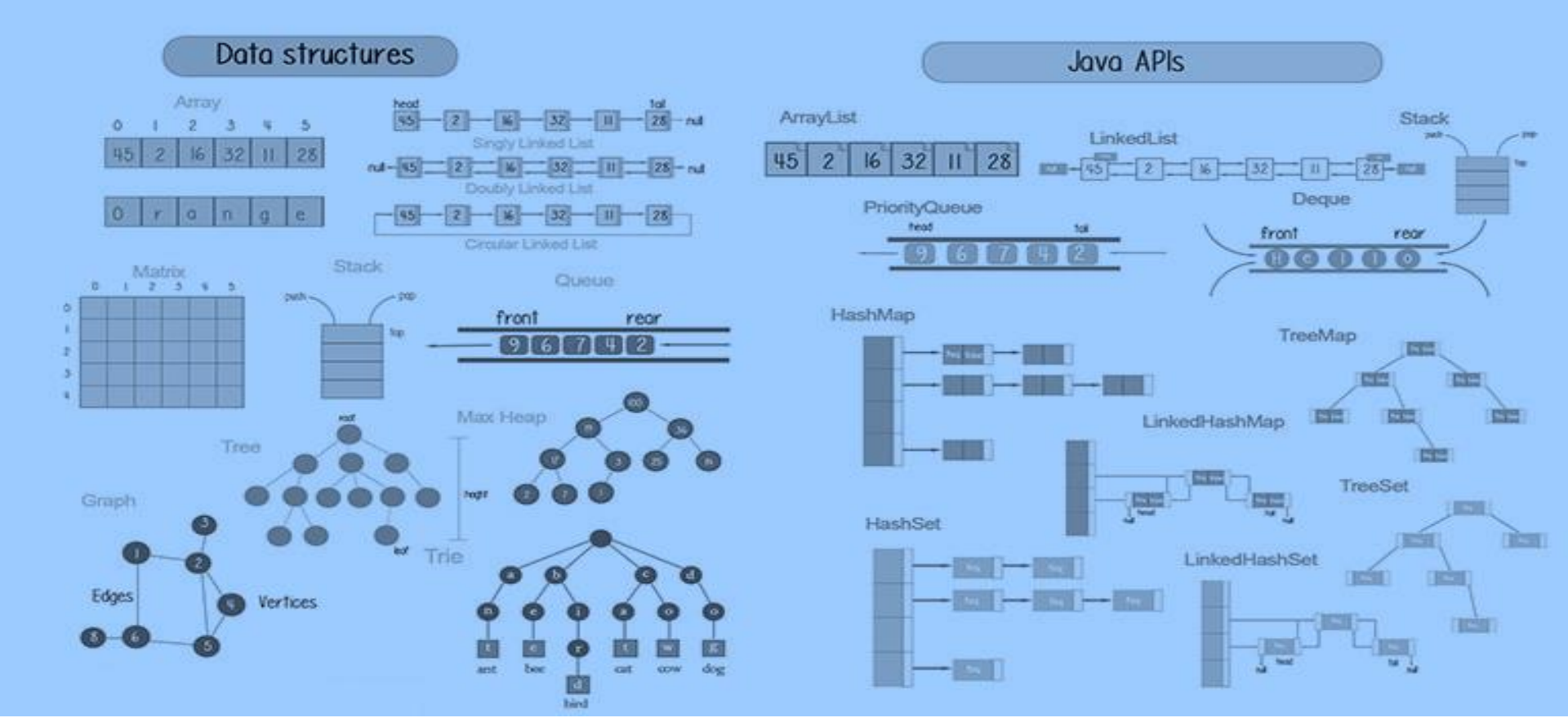


Java Programming

CPT111 – Lecture 11

Teng Ma

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lecture 11

List, Set, Map

Welcome!

- Welcome to Lecture 11!
- Last week, we have learned about File I/O and Graphical UI!
  - create, read and write File
  - Graphical interactions
- In this lecture we are going to learn about
  - Lists
    - creating, accessing and manipulating a List as ArrayList and LinkedList
    - using list in method and in a class as an instance variable
  - Set and Map
    - Characteristics and Operations supported by Set and Map
    - Using Set and Map to do problem solving

Welcome!

- Learning outcomes covered in this lecture:
  - **A. Understand and appreciate the principles of using object oriented programming techniques for the construction of professional robust, maintainable programs deployed to meet real world business goals;**
  - **B. Design, write, compile, test, debug and execute Java programs using an integrated development environment (IDE);**
  - C. Be able to develop programs that the user can interact with either through graphical representation, or text based menus;
  - **D. Use Java programming to represent, display, and manipulate data as objects while being cognizant of associated risk/safety for safe operation of computing and information systems.**
  - E. Have an awareness of essential legal, social, ethical & professional issues related to software development

Part 1: List, ArrayList and LinkedList

- In the first part of lecture, we will learn about lists
  - how to create a List
  - how to add and access its data
  - how to insert and remove its data
  - how to use it for problem-solving

Array Deficiencies

- Say you want to create an array of Swordsman objects
- What if you don't know how many objects to store beforehand?
  - arrays need to declared with a size!
- What if you want to store any numbers of objects?
  - arrays cannot grow arbitrarily!

Java Collections and List

- Java provides a number of more powerful and flexible tools for managing collections of objects: the **Java Collections Framework**
- It includes List (first part) and Set, Map (second part)
  - *both would be very useful for your CW3*

- A List contains an ***ordered*** collection of zero or more objects, where the same object might appear multiple times
  - we can add and remove items to and from the List, which will grow and shrink to accommodate its contents

Creating Lists (1)

- Java helps us distinguish between the *specification* of a type – *what does it do?* – and the *implementation* – *what is the code?*
- List, Set and Map are all *interfaces*: they define **how** these respective types work, but they **don't** provide implementation code
  - *will learn more in CPT204 Advanced OOP later*

- One advantage is that we, the users of these types, get to choose different implementations in different situations

- To create Lists, specify the object/generic type inside diamond <> operator:
  - `List<String> list1 = new ArrayList<String>();`
  - `List<String> list2 = new LinkedList<String>();`

Creating Lists (2)

- If the generic type parameters are the **same** on the left and right, Java can infer what's going on and save us some typing:
  - `List<String> list1 = new ArrayList<>();`
  - `List<String> list2 = new LinkedList<>();`

empty here. let's use this!

- `ArrayList` and `LinkedList` are two implementations of List
  - both provide all the operations of List, and those operations must work as described in the documentation for List
  - `list1` and `list2` will behave the same way,
    i.e. if we swap which one used `ArrayList` vs `LinkedList`,
    our code will **not** break

In-Class Quiz 11.2

- What do you think the resulting list would be after these list operations?

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
list.add(1, "X");
list.set(2, "Y");
System.out.println(list);
```

- ⊗ [A X Y C]
- ○ [A B X Y C]
- ○ [A X B Y C]
- ○ [A X B C Y]

List Operations

- To declare a List variable, for example of integers, with an ArrayList:
  - `List<Integer> list = new ArrayList<>();`
- Some common operations:
  - adding `list.add(5)`
  - inserting `list.add(1, 3)`
  - assigning `list.set(2, 7)`
  - retrieving `list.get(2)`
  - length `list.size()`
  - remove `list.remove(1)`
  - remove all `list.clear()`
- Create a list from an array

  `List<Integer> list = Arrays.asList(10, 20, 30)`

wrapper class for primitives

adding to the back of list

replace the value at that index

List and ArrayList

```
List<Integer> list = new ArrayList<Integer>();
```

- `List` is an **interface**, a type that can't be constructed directly with `new`, but that instead *specifies the operations* that a List must provide
  - `ArrayList` is a **class**, a concrete type that provides *implementations of those operations*
  - `ArrayList` isn't the only implementation of the `List` type, though it's the most commonly used one
    - `LinkedList` is another implementation

Object type / generic type parameter

```
List<Integer> list = new ArrayList<Integer>();
```

- We wrote `List<Integer>` instead of `List<int>`
  Lists only know how to deal with *object types*, not *primitive types*
  - In Java, each of the primitive types (lowercase, abbreviated) has an equivalent object type (capitalized, fully spelled out)
  - Java requires us to use these object type equivalents when we **parameterize** a type with diamond operator **< >** angle brackets

ArrayList vs LinkedList

- Two possible implementations of a List:

	`ArrayList`	`LinkedList`
Actual data structure	Resizing Array	Doubly Linked List
Storing and accessing	Fast	Slow
Inserting and deleting	Slow	Fast

you will actually create both yourself from scratch in CPT204!

Iterate through List

- You can use the **enhanced for loops** to iterate through a list
  - for example:

```
for (int x : list) {
    System.out.print(x + " ");
}
```

- Demo examples:
  - Write a function `findMaxInt` to find the maximum element in an input `List` of integers using the enhanced for loop
  - Write a function `readFile` that takes a file name and returns a `List` of `Swordsman` objects generated from the file
  - Write a `DemonSlayerCorps` class that uses a `List` to store a troops of `Swordsman` objects

FindMaxInt

- Write a function `findMaxInt` to find the maximum element in an input `List` of integers using the enhanced for loop

```
public static int findMaxInt(List<Integer> list) {
    int max = list.get(0);
    for (int num : list) {
        if (num > max) {
            max = num;
        }
    }
    return max;
}
```

Reading from a file and returning List of Swordsman

- Write a function `readFile()` that takes a file name and returns a `List` of `Swordsman` objects generated from the file!
  - the file contains names of the `Swordsman`

```
public static List<Swordsman> readFile(String fileName) {
    File file = new File(fileName);
    List<Swordsman> list = new ArrayList<>();
    try {
        Scanner input = new Scanner(file);
        while (input.hasNextLine()) {
            String name = input.nextLine();
            Swordsman swordsman = new Swordsman(name);
            list.add(swordsman);
        }
        input.close();
    } catch (IOException ioe) {
        System.out.println(ioe.getMessage());
    }
    return list;
}
```

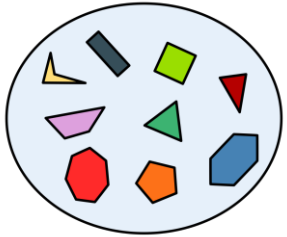
DemonSlayerCorps Class

- Write a `DemonSlayerCorps` class that uses a `List` to store a troops of `Swordsman` objects!
  - complete it with constructors and instance methods such as `heal()` that heals the whole troops

Part 2: Set and Map

- In the second part of lecture, we will learn about Set and Map
- We continue on **collection**
- Java provides a number of powerful and flexible tools for managing collections of objects: the **Java Collections Framework**
- It includes List, **Set and Map**
  - *both would be very useful for your CW3*

Set



- A `Set` stores a unordered group of zero or more unique objects
  - no sense of order: either an item is in the set, or not
  - no duplicates: an object cannot appear in a `Set` multiple times

- Two popular `Set` implementations in Java:
  - `HashSet` – implemented using a hash table
  - `TreeSet` – implemented using a self-balancing tree data structure

- Other implementations: `LinkedHashSet` and `EnumSet`
- Will learn more in Data Structure and Advanced OOP courses

Set Operations

- To declare a `Set`, for example of integers, with a `HashSet`:
  - `Set<Integer> set = new HashSet<>();`
- Some common operations:
  - adding `set.add(5)`
  - checking membership `set.contains(5)`
  - remove `set.remove(5)`
  - union `set.addAll(c)`
  - intersection `set.retainAll(c)`
  - difference `set.removeAll(c)`
  - contain/subset `set.containsAll(c)`
  - size `set.size()`
  - clear contents `set.clear()`

c: other collection, e.g. other set

adding (no concepts of order)

present or not

set ∪ c

set ∩ c

set \ c

check whether set ⊇ c

Iterate through Set

- You can use the **enhanced for loops/for each loop** to iterate through a Set

```
public static void main(String[] args) {

    Set<String> set = new HashSet<>();
    set.add("B");
    set.add("C");
    set.add("A");

    System.out.println(set);

    for (String string : set) {
        System.out.println(string);
    }
}
```

NetBeans Shortcut

En	Enumeration
Ex	Exception
Ob	Object
Psf	public static final
Psfb	public static final boolean
Psfi	public static final int
Psfs	public static final String
St	String
ab	abstract
as	assert true;
bcom	/**/
bo	boolean
br	break;
ca	catch (
cl	class
cn	continue
db	double
df	default:
dowhile	do { } while (*condition*);
eq	equals
ex	extends
fa	false
fcom	// <editor-fold></editor-fold>
fi	final
fl	float

forc	for (Iterator *it* = *collection*.iterator();
	it.hasNext();) {
	Object elem = (*Object*) *it*.next();
	}

fore	for (*Object elem* : iterable) { }
fori	for (int *i* = 0; i < *arr*.length; *i*++) {}
forl	for (int i = 0; i < lst.size(); i++) { Object object = lst.get(i); }
forst	for (StringTokenizer st = new StringTokenizer(""); st.hasMoreTokens();) }
forv	for (int i = 0; i < vct.size(); i++) { *Object* object = vct.elementAt(i);}
fy	finally{ |}
ie	interface
ifelse	if (*condition*){}else { }
iff	if (exp) {}
im	implements
inst	if (*exp* instanceof *Object*) { *Object* obj = (*Object*) exp;
iof	instanceof
ir	inport
le	length
na	native
newo	*Object name* = new *Object*(*args*);
pe	protected
pr	private
psf	private static final
psfb	private static final boolean
psfi	private static final int
psfs	private static final String
pst	printStackTrace();
psvm	public static void main(String[] args){ }
pu	public
re	return
runn	Runnable runnable = new Runnable() { public void run() {}};
serr	System.err.println ("|");
sh	short
sout	System.out.println ("|");
soutv	System.out.println("*Object* = " + *Object*);
st	static
su	super
sw	switch (var) { case val: break; default: throw new AssertionError();}
sy	synchronized
tds	Thread.dumpStack();
th	throws
tr	transient
trycatch	try {} catch (Exception *e*) {}
tw	throw
vo	volatile
wh	while (
whileit	while (*it*.hasNext()) { *Object elem* = (*Object*) *it*.next();}
whilen	while (*en*.hasMoreElements()) { *Object elem* = (*Object*) *en*.nextElement();}
whilexp	while (exp) {}

In-Class Quiz 12.1

- What will be printed when we check membership of "D" ?

```
public static void main(String[] args) {

    Set<String> set = new HashSet<>();
    set.add("B");
    set.add("C");
    set.add("A");

    System.out.println(set.contains("D"));

}
```

- ○ null
- ⊗ false
- ○ there is an exception message printed

Set Operations – Example 1

- Creating set from list or other set, union, test containment, intersection:

```
public static void main(String[] args) {

    Set<Integer> set1 = new HashSet<>();
    set1.addAll(Arrays.asList(new Integer[] {1, 3}));
    Set<Integer> set2 = new HashSet<>();
    set2.addAll(Arrays.asList(new Integer[] {2, 5, 4}));

    set1.addAll(set2);
    System.out.println("set1 = set1 ∪ set2 = " + set1);

    System.out.println(set1.containsAll(set2));

    Set<Integer> intersectionSet = new HashSet<>(set1);
    Set<Integer> set3 = new HashSet<Integer>();
    set3.addAll(Arrays.asList(new Integer[] {1, 5, 6, 7}));
    intersectionSet.retainAll(set3);
    System.out.println("set1 ∩ set3 = " + intersectionSet);
}
```

creating set from list (that's created from array)

creating set from another set

Set Operations – Example 2

- Set difference and remove an element:

```
public static void main(String[] args) {

    Set<Integer> set4 = new HashSet<>();
    set4.addAll(Arrays.asList(new Integer[] {1, 2, 3, 4, 5}));
    Set<Integer> set5 = new HashSet<>();
    set5.addAll(Arrays.asList(new Integer[] {2, 5}));

    Set<Integer> setDifference = new HashSet<>(set4);
    setDifference.removeAll(set5);
    System.out.println("set4 \\ set5 = " + setDifference);

    setDifference.remove(1);
    System.out.println(setDifference);
}
```

Map

- A **map** stores key/value pairs, where each **key** has an associated **value**
  - Given a particular key, the map can look up the associated value *very quickly* !
- A map is also known as a **dictionary** or a **symbol table**
- Implementation of maps are `TreeMap`, `HashMap` and `LinkedHashMap`
  - Will learn more in Data Structure and Advanced OOP courses

Map Operations

- Operations in maps include:
  - `map.put(key, val)` adding the mapping key → val
  - `map.get(key)` get the value for a key
  - `map.containsKey(key)` test whether the map has the key
  - `map.containsValue(value)` test whether the map has the value
  - `map.remove(key)` delete a mapping
  - `map.keySet()` returns a set containing all keys

In-Class Quiz 12.2

- What will be printed when we look up "Kyojuro" ?

```
Swordsman tanjiro = new Swordsman("Tanjiro", 5);
Swordsman zenitsu = new Swordsman("Zenitsu", 2);
Swordsman inosuke = new Swordsman("Inosuke", 10);

Map<String, Swordsman> troops = new HashMap<>();

troops.put("Tanjiro", tanjiro);
troops.put("Zenitsu", zenitsu);
troops.put("Inosuke", inosuke);

Swordsman swordsman = troops.get("Kyojuro");
System.out.println(swordsman);
```

- ⊗ null
- ○ false
- ○ empty string / nothing is printed
- ○ there is an exception message printed

Map Operations Examples and Iteration

```
Map<String, Swordsman> troops = new HashMap<>();

troops.put("Tanjiro", tanjiro);
troops.put("Zenitsu", zenitsu);
troops.put("Inosuke", inosuke);

Swordsman swordsman;
swordsman = troops.get("Kyojuro");

if (troops.containsKey("Tanjiro")) {
    swordsman = troops.get("Tanjiro");
    // do something with swordsman
}

for (String key : troops.keySet()) {
    System.out.println(key + ": " + troops.get(key));
}
```

create an empty map

we can iterate over the keys of a map

Map Example : NumWords

- Suppose we have a list of strings of multiple duplicate words, such as ["a", "b", "a", "c", "b", "word", "a", "word"]
- We want to find out which words appear in the list and how many times each one appears
- Here is an algorithm NumWords using Map:
  - Create a Map<String, Integer>
  - Loop through all the strings in the list
  - Use each string as a key into the map
  - Use the Integer value to store the number of times that string has been seen
  - There are 2 cases:
    - The first time we see a word, it is not yet in the map, we add it
    - The later times we see the word, it is already in the map, we update it

NumWords Code

```
import java.util.HashMap;
import java.util.Map;

public class NumWords {
    public Map<String, Integer> wordCount(List<String> list) {
        Map<String, Integer> map = new HashMap<>();
        int num;
        for (String word : list) {
            if (!map.containsKey(word)) {
                map.put(word, 1);
            }
            else {
                num = map.get(word);
                map.put(word, num + 1);
            }
        }
        return map;
    }
}
```

the first time we've seen string `word`

we increment the number of `word`

Thank you for your attention !

- In this lecture, you have learned:
  - how to create, access and manipulate a List implementation with ArrayList and LinkedList
  - how to use Set and Map to solve problems

- Please continue to Lab 11 to complete Lab Tasks, and then solve
  - Exercise and CW1 of Week #11